FIAP · Faculdade de Informática e Administração Paulista · Engenharia de Software

# Conecta Alimento

API RESTful em Java com Spring Boot e Oracle

**Global Solution 2026.2 · Domain Driven Design - Java**


## Integrantes

| Nome completo | RM |
|---|---|
| Guilherme Barone Milani | RM562114 |
| Guilherme Frangione Ferreira Acácio | RM562475 |
| André Dominicis Monteiro | RM562397 |
| Luigi Escudero Grigoletto | RM562505 |
| Luis Gustavo Mattos da Silva | RM561672 |


Plataforma que conecta estabelecimentos com excedentes alimentares a instituições sociais no piloto Bom Retiro, Pari e Brás (região central de São Paulo).


São Paulo - SP

Novembro de 2026

# Sumário


**1. Descritivo da solução** ..... **3**

1.1 Contexto e problema ..... 3

1.2 A solução: Conecta Alimento API ..... 3

1.3 Fluxo da doação e estados ..... 3

1.4 Funcionalidades implementadas ..... 4

1.5 Escopo e limites ..... 4

**2. Justificativa e objetivos** ..... **4**

2.1 Justificativa ..... 4

2.2 Objetivo geral ..... 4

2.3 Objetivos específicos ..... 5

2.4 Relação com CitizenTech e ODS ..... 5

**3. Arquitetura e tecnologias** ..... **6**

3.1 Camadas da aplicação ..... 6

3.2 Orientação a objetos no código ..... 6

**4. Diagrama de classes** ..... **7**

4.1 Domínio (pacote model) ..... 7

4.2 Camadas controller, bo e dao ..... 8

4.3 Exceções e configuração ..... 9

**5. Protótipos de tela** ..... **10**

**6. Endpoints da API** ..... **11**

6.1 Tabela de endpoints ..... 11

6.2 Códigos de resposta e tratamento de exceções ..... 13

**7. Modelo de dados (DER)** ..... **14**

**8. Instruções de execução** ..... **15**

8.1 Ferramentas e versões ..... 15

8.2 Links do projeto ..... 15

8.3 Executar localmente no IntelliJ ..... 15

8.4 Deploy no Render ..... 16

8.5 Como testar ..... 16

# 1. Descritivo da solução

## 1.1 Contexto e problema

Estabelecimentos como padarias, mercados, restaurantes e hortifrútis podem ter alimentos excedentes em boas condições de consumo, mas nem sempre encontram rapidamente uma instituição capaz de recebê-los. Do outro lado, ONGs, cozinhas comunitárias e abrigos têm dificuldade para descobrir quais alimentos estão disponíveis por perto e dentro do prazo de retirada. Sem um canal organizado, a comunicação fica espalhada em conversas e contatos pessoais, e oportunidades de doação se perdem.

O problema envolve, ao mesmo tempo: **disponibilidade** (o que existe e em que quantidade), **prazo** (até quando pode ser retirado), **localização** (aproximar as partes em um território viável), **logística** (combinar data e horário), **segurança alimentar** (conservação, armazenamento, integridade e validade) e **rastreabilidade** (histórico da solicitação e da confirmação da entrega).

## 1.2 A solução: Conecta Alimento API

A **Conecta Alimento API** é a implementação em Java da capacidade central do projeto CitizenTech da equipe: **conectar a oferta de excedentes alimentares à demanda de instituições sociais** e acompanhar cada doação do anúncio até a confirmação do recebimento. É uma API RESTful desenvolvida com **Spring Boot**, persistência no **banco Oracle da FIAP** via JDBC e organização em camadas (model, dao, bo, controller, exception), publicada no **Render** para ser consumida pelo aplicativo (protótipos da seção 5).

A API tem dois perfis de usuário, modelados por herança a partir da classe abstrata `Usuario`: **estabelecimento doador** (cadastra excedentes e decide sobre solicitações) e **instituição receptora** (busca, solicita, retira e confirma). Os alimentos também são polimórficos: `AlimentoPerecivel` exige condição de armazenamento compatível com a categoria (laticínios e carnes precisam de refrigeração), e `AlimentoNaoPerecivel` exige embalagem íntegra. A busca calcula a distância (fórmula de Haversine) entre a instituição e cada doador para ordenar as ofertas mais próximas.

## 1.3 Fluxo da doação e estados

O ciclo de uma doação é controlado pelas regras de negócio (camada `bo`) e pelos próprios objetos de domínio, que impedem transições inválidas de status:

1. **Publicação** - o estabelecimento cadastra o alimento com quantidade, validade, janela de retirada e declaração de ciência das orientações de segurança → `DISPONIVEL`.
2. **Busca e solicitação** - a instituição filtra por categoria, bairro, prazo e distância e solicita uma quantidade → alimento `SOLICITADO`, solicitação `PENDENTE`.
3. **Análise** - o estabelecimento aceita (alimento `RESERVADO`; demais pedidos pendentes ficam `NAO_ATENDIDA`) ou recusa com motivo.
4. **Agendamento** - a retirada é marcada dentro da janela e no futuro → `RETIRADA_AGENDADA`; pode ser reagendada ou cancelada.
5. **Confirmação** - entrega (estabelecimento) e recebimento (instituição) são confirmados → `CONCLUIDO`; a doação passa a compor o histórico e os indicadores.

Alimentos cujo prazo passou sem conclusão viram `EXPIRADO`; a remoção é lógica (`REMOVIDO`), preservando o histórico.

### 1.4 Funcionalidades implementadas

- CRUD completo de estabelecimentos e instituições (e-mail e CNPJ únicos, senha com hash SHA-256)
- Login por e-mail e senha com identificação do perfil
- Cadastro, edição e remoção lógica de alimentos, com validação de segurança alimentar
- Busca de doações com filtros e ordenação por distância
- Orientações de segurança alimentar gerais e por tipo de alimento
- Solicitação, alteração de quantidade, aceite, recusa e cancelamento
- Agendamento, reagendamento e cancelamento da retirada
- Confirmação de entrega e de recebimento
- Indicadores de impacto: doações concluídas, kg doados, % retiradas, tempo médio, instituições atendidas
- Tratamento de erros padronizado e documentação interativa (Swagger)

### 1.5 Escopo e limites

A plataforma organiza o processo de doação; ela **não certifica alimentos** nem substitui orientações sanitárias. A responsabilidade pela qualidade do alimento continua sendo do doador, que declara ciência das orientações a cada publicação. O piloto considera apenas os bairros Bom Retiro, Pari e Brás, e os dados de demonstração carregados pela API são fictícios.

## 2. Justificativa e objetivos

### 2.1 Justificativa

Trata-se de uma necessidade social concreta, ligada ao cotidiano de cidadãos, estabelecimentos e instituições, com um recorte tecnológico viável: organizar informações e etapas de um processo que hoje acontece de forma dispersa. Um fluxo único e padronizado reduz o atrito entre quem tem o alimento e quem precisa dele, sem apresentar a tecnologia como substituta de políticas públicas ou de responsabilidades sanitárias.

Do ponto de vista técnico, a escolha por uma **API RESTful** separa as regras de negócio da interface: o mesmo serviço pode atender o aplicativo móvel, um painel web e futuras integrações. A modelagem orientada ao domínio (entidades, objetos de valor, herança e polimorfismo) deixa as regras próximas dos conceitos do problema - por exemplo, é o próprio `AlimentoPerecivel` que sabe validar sua refrigeração - e torna o sistema extensível para novos tipos de alimento ou de perfil.

O recorte territorial (Bom Retiro, Pari e Brás) permite distâncias curtas, prazos de retirada realistas e perfis variados de comércio e atendimento social, adequados para um piloto.

### 2.2 Objetivo geral

Implementar em Java, com Spring Boot e banco Oracle, uma API RESTful que represente o domínio do Conecta Alimento e viabilize, de ponta a ponta, a doação de excedentes alimentares entre estabelecimentos e instituições sociais.

## 2.3 Objetivos específicos

- Modelar o domínio com classes, herança, polimorfismo (sobrescrita e sobrecarga) e encapsulamento.
- Persistir os dados no Oracle da FIAP com uma camada DAO em JDBC e tabelas compatíveis com o modelo.
- Concentrar as regras de negócio na camada BO: segurança alimentar, quantidades, janelas de retirada e transições de status.
- Expor os recursos por controllers REST com os métodos GET, POST, PUT e DELETE.
- Padronizar o tratamento de exceções com respostas JSON e códigos HTTP adequados.
- Calcular indicadores de impacto previstos no projeto (kg doados, doações concluídas, % retiradas, tempo médio).
- Publicar a API no Render, com documentação Swagger para testes.

## 2.4 Relação com CitizenTech e ODS

O projeto se relaciona ao tema **CitizenTech** porque usa tecnologia para coordenar atores locais em torno de uma necessidade social situada em um território, apoiando decisões práticas: o que está disponível, onde está, até quando pode ser retirado e quem é responsável pela próxima etapa. Também dialoga com os ODS **2 - Fome Zero e Agricultura Sustentável**, **11 - Cidades e Comunidades Sustentáveis** e **12 - Consumo e Produção Responsáveis**.

# 3. Arquitetura e tecnologias

## 3.1 Camadas da aplicação

O fluxo de uma requisição segue **Controller → BO → DAO → Oracle**. Erros em qualquer camada sobem como exceções da aplicação e são convertidos em JSON pelo `GlobalExceptionHandler`.

| Pacote | Responsabilidade |
|---|---|
| `model` | Entidades do domínio (`Entidade`, `Usuario`, `EstabelecimentoDoador`, `InstituicaoReceptora`, `Alimento`, `AlimentoPerecivel`, `AlimentoNaoPerecivel`, `Solicitacao`, `Retirada`), o objeto de valor `Endereco` e as enumerações. |
| `dao` | Acesso ao Oracle com JDBC (`PreparedStatement`): `BaseDAO<T>` genérico com consultas comuns e um DAO por tabela. |
| `bo` | Regras de negócio e transações (`@Transactional`): validações, fluxo de status, distância e indicadores. |
| `controller` | Endpoints REST (GET, POST, PUT, DELETE), validação da entrada e códigos HTTP. |
| `dto` | Corpos de requisição e resposta (records) com Bean Validation. |
| `exception` | Hierarquia `ApiException` e o tratador global de erros. |
| `config` | CORS, Swagger/OpenAPI e criação automática das tabelas e dos dados de demonstração. |

## 3.2 Orientação a objetos no código

| Conceito | Onde aparece no código |
|---|---|
| Identificador único | Todas as entidades herdam `id` e `dataCadastro` de `Entidade` (chave primária IDENTITY no Oracle). |
| Herança de atributos, métodos e construtores | `Entidade → Usuario → EstabelecimentoDoador / InstituicaoReceptora`; `Entidade → Alimento → AlimentoPerecivel / AlimentoNaoPerecivel`; `BaseDAO<T>` → DAOs; `UsuarioBO` → `EstabelecimentoBO` / `InstituicaoBO`; `ApiException` → 5 exceções. Construtores das subclasses chamam `super(...)`. |
| Polimorfismo - sobrescrita (override) | `resumo()`, `getTipoPerfil()`, `getTipo()`, `getCondicaoArmazenamento()`, `validarSegurancaAlimentar()`, `orientacoesEspecificas()`, `getStatus()` das exceções, `mapear()` / `completar()` dos DAOs, `toString()`, `equals()`, `hashCode()`. |
| Polimorfismo - sobrecarga (overload) | `distanciaAte(Endereco)` / `distanciaAte(lat, lon)`; `atualizarContato(tel)` / `atualizarContato(tel, endereco)`; `estaExpirado()` / `estaExpirado(ref)`; `recusar()` / `recusar(motivo)`; `confirmarRecebimento(q)` / `confirmarRecebimento(q, obs)`; `buscarDisponiveis()` com e sem filtros; construtores padrão e com parâmetros. |
| Abstração e encapsulamento | Classes e métodos abstratos; atributos privados com getters e setters; senha guardada apenas como hash (`@JsonIgnore`). |

# 4. Diagrama de classes

## 4.1 Domínio (pacote model)

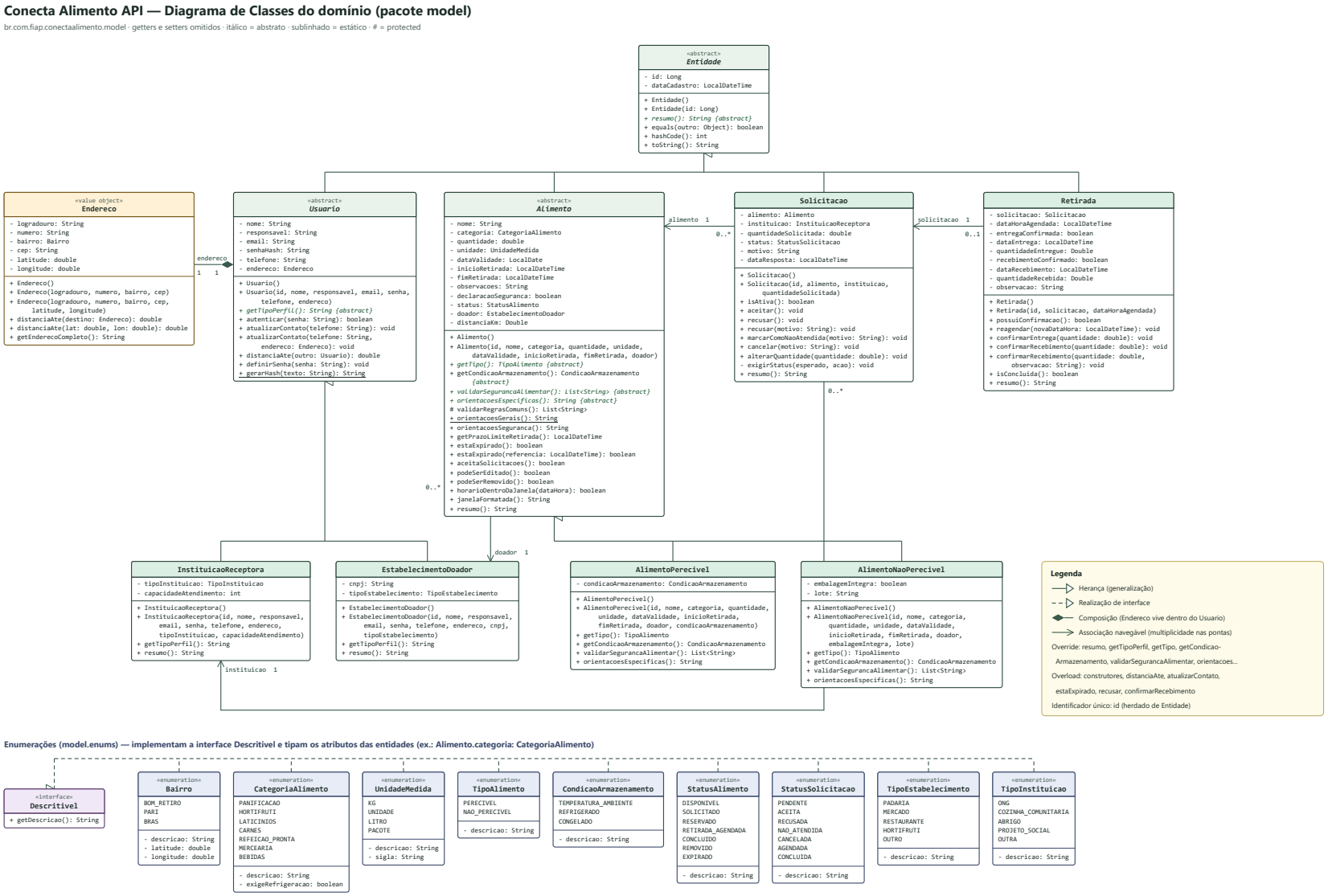



Figura 1 - Diagrama de classes do domínio. Getters e setters omitidos. Os arquivos SVG/PNG em alta resolução estão na pasta `docs/diagramas` do repositório.

## 4.2 Camadas controller, bo e dao

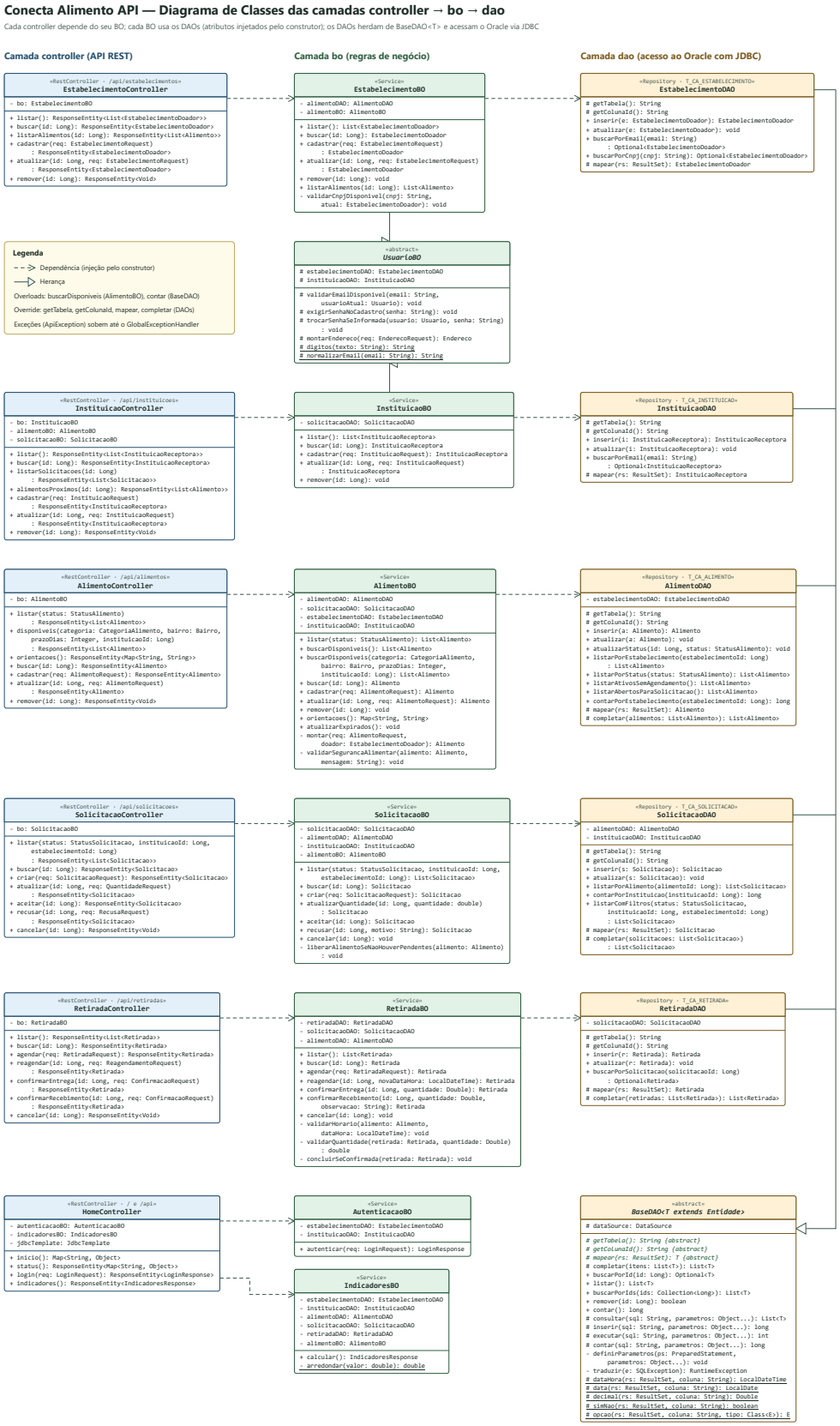



Figura 2 - Controllers, BOs e DAOs com as dependências injetadas e a herança de `UsuarioBO` e `BaseDAO<T>`.

## 4.3 Exceções e configuração

### Conecta Alimento API — Diagrama de Classes: exceções e configuração

pacotes exception e config · cada exceção sobrescreve getStatus() (polimorfismo) e o GlobalExceptionHandler converte tudo em ErroResposta (JSON)

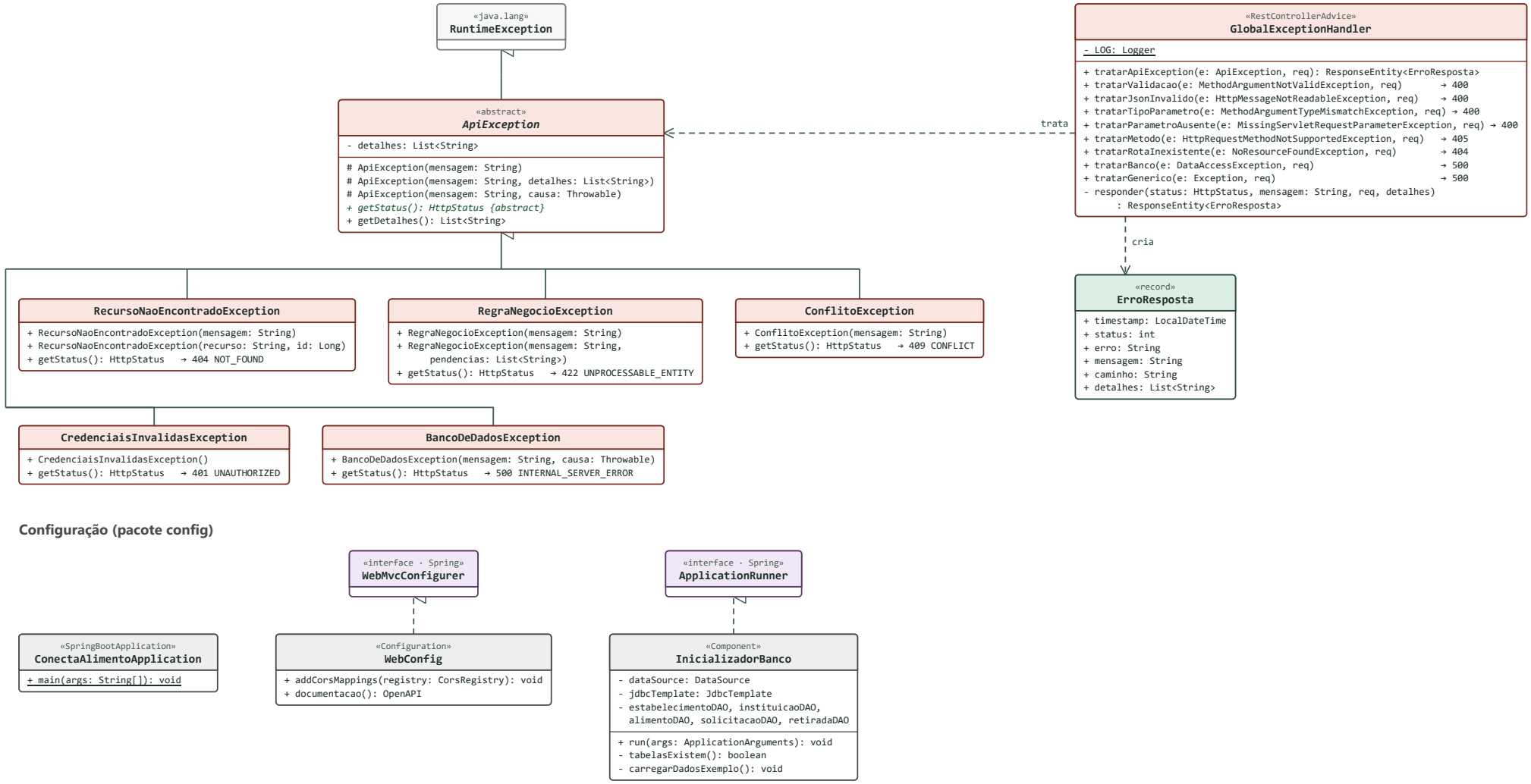
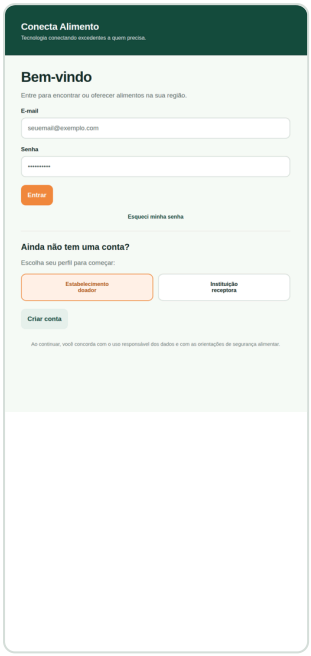



Figura 3 - Hierarquia de exceções (cada uma define seu status HTTP), tratador global e classes de configuração.

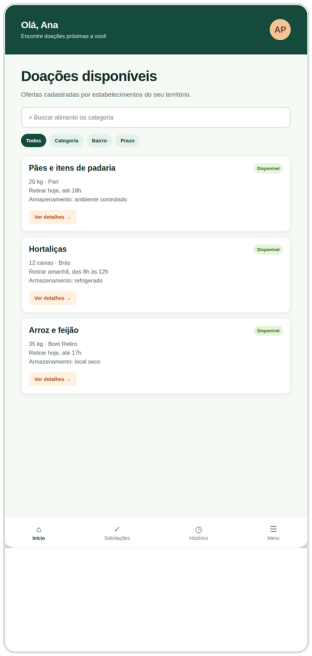
# 5. Protótipos de tela

Telas do aplicativo que consome a API. As duas primeiras vêm do protótipo da disciplina de Agile; as demais detalham o fluxo completo. Abaixo de cada tela estão os endpoints usados.




**Entrada e cadastro**

`POST /api/login` · `POST /api/estabelecimentos` · `POST /api/instituicoes`




**Doações disponíveis**

`GET /api/alimentos/disponiveis`




**Nova doação (estabelecimento)**

`POST /api/alimentos`




**Detalhe e solicitação (instituição)**

`GET /api/alimentos/{id}` · `POST /api/solicitacoes`




**Solicitações recebidas (estabelecimento)**

`GET /api/solicitacoes` · `PUT …/{id}/aceitar` · `PUT …/{id}/recusar`




**Agendamento e confirmação da retirada**

`POST /api/retiradas` · `PUT /api/retiradas/{id}` · `PUT …/confirmar-recebimento`




**Impacto no território**

`GET /api/indicadores`

# 6. Endpoints da API

## 6.1 Tabela de endpoints

Base: `http://localhost:8080`. Corpos em JSON; datas no formato ISO (`2026-11-20` e `2026-11-20T18:00:00`). POST de criação devolve **201** com o cabeçalho `Location`; DELETE devolve **204**.

| Método | Endpoint | Descrição | Corpo | Respostas |
|---|---|---|---|---|
| **Geral** | | | | |
| GET | / | Informações da API e lista de recursos | – | 200 |
| GET | /api/status | Status da API e da conexão com o Oracle | – | 200, 503 |
| POST | /api/login | Autentica estabelecimento ou instituição e devolve o perfil | LoginRequest | 200, 400, 401 |
| GET | /api/indicadores | Indicadores de impacto (doações, kg, % retiradas, tempo médio) | – | 200 |
| **Estabelecimentos doadores** | | | | |
| GET | /api/estabelecimentos | Lista os estabelecimentos | – | 200 |
| GET | /api/estabelecimentos/{id} | Busca um estabelecimento | – | 200, 404 |
| GET | /api/estabelecimentos/{id}/alimentos | Alimentos publicados pelo estabelecimento | – | 200, 404 |
| POST | /api/estabelecimentos | Cadastra estabelecimento (e-mail e CNPJ únicos) | EstabelecimentoRequest | 201, 400, 409 |
| PUT | /api/estabelecimentos/{id} | Atualiza os dados (senha opcional) | EstabelecimentoRequest | 200, 400, 404, 409 |
| DELETE | /api/estabelecimentos/{id} | Remove, se não houver alimentos cadastrados | – | 204, 404, 409 |
| **Instituições receptoras** | | | | |
| GET | /api/instituicoes | Lista as instituições | – | 200 |
| GET | /api/instituicoes/{id} | Busca uma instituição | – | 200, 404 |
| GET | /api/instituicoes/{id}/solicitacoes | Solicitações feitas pela instituição | – | 200, 404 |
| GET | /api/instituicoes/{id}/alimentos-proximos | Alimentos disponíveis ordenados por distância (km) | – | 200, 404 |
| POST | /api/instituicoes | Cadastra instituição (e-mail único) | InstituicaoRequest | 201, 400, 409 |
| PUT | /api/instituicoes/{id} | Atualiza os dados (senha opcional) | InstituicaoRequest | 200, 400, 404, 409 |
| DELETE | /api/instituicoes/{id} | Remove, se não houver solicitações | – | 204, 404, 409 |

| | | | | |
|---|---|---|---|---|
| **Alimentos (doações)** | | | | |
| GET | `/api/alimentos?status=` | Lista alimentos (filtro opcional por status) | – | 200, 400 |
| GET | `/api/alimentos/disponiveis?categoria=&bairro=&prazoDias=&institucaoId=` | Busca para a instituição: filtros e distância | – | 200, 400, 404 |
| GET | `/api/alimentos/orientacoes` | Orientações de segurança alimentar | – | 200 |
| GET | `/api/alimentos/{id}` | Busca um alimento | – | 200, 404 |
| POST | `/api/alimentos` | Publica alimento perecível ou não perecível | `AlimentoRequest` | 201, 400, 404, 422 |
| PUT | `/api/alimentos/{id}` | Edita (enquanto disponível ou solicitado) | `AlimentoRequest` | 200, 400, 404, 422 |
| DELETE | `/api/alimentos/{id}` | Remoção lógica (status REMOVIDO) e cancela pedidos ativos | – | 204, 404, 422 |
| **Solicitações** | | | | |
| GET | `/api/solicitacoes?status=&institucaoId=&estabelecimentoId=` | Lista com filtros opcionais | – | 200, 400 |
| GET | `/api/solicitacoes/{id}` | Busca uma solicitação | – | 200, 404 |
| POST | `/api/solicitacoes` | Instituição solicita um alimento | `SolicitacaoRequest` | 201, 400, 404, 409, 422 |
| PUT | `/api/solicitacoes/{id}` | Altera a quantidade (somente pendente) | `QuantidadeRequest` | 200, 400, 404, 422 |
| PUT | `/api/solicitacoes/{id}/aceitar` | Estabelecimento aceita; alimento fica reservado | – | 200, 404, 422 |
| PUT | `/api/solicitacoes/{id}/recusar` | Estabelecimento recusa (motivo opcional) | `RecusaRequest` | 200, 404, 422 |
| DELETE | `/api/solicitacoes/{id}` | Instituição cancela a solicitação | – | 204, 404, 422 |
| **Retiradas** | | | | |
| GET | `/api/retiradas` | Lista as retiradas | – | 200 |
| GET | `/api/retiradas/{id}` | Busca uma retirada | – | 200, 404 |
| POST | `/api/retiradas` | Agenda a retirada dentro da janela do alimento | `RetiradaRequest` | 201, 400, 404, 409, 422 |
| PUT | `/api/retiradas/{id}` | Reagenda a retirada | `ReagendamentoRequest` | 200, 400, 404, 422 |
| PUT | `/api/retiradas/{id}/confirmar-entrega` | Estabelecimento confirma a entrega | `ConfirmacaoRequest` | 200, 404, 422 |
| PUT | `/api/retiradas/{id}/confirmar-recebimento` | Instituição confirma o recebimento | `ConfirmacaoRequest` | 200, 404, 422 |
| DELETE | `/api/retiradas/{id}` | Cancela o agendamento (volta para aceita) | – | 204, 404, 422 |

### Exemplo - publicar um alimento

```
POST /api/alimentos
{ "estabelecimentoId": 1, "nome": "Pão de forma", "tipo": "NAO_PERECIVEL", "categoria":
"PANIFICACAO",
  "quantidade": 10, "unidade": "PACOTE", "dataValidade": "2026-11-20", "fimRetirada": "2026-11-
14T18:00:00",
  "embalagemIntegra": true, "lote": "L123", "declaracaoSeguranca": true }

201 Created   Location: /api/alimentos/13
{ "id": 13, "nome": "Pão de forma", "categoria": "PANIFICACAO", "quantidade": 10.0, "unidade":
"PACOTE",
  "status": "DISPONIVEL", "tipo": "NAO_PERECIVEL", "condicaoArmazenamento": "TEMPERATURA_AMBIENTE",
  "prazoLimiteRetirada": "2026-11-14T18:00:00", "doador": { "id": 1, "nome": "Padaria Santos", ... }
}
```

## 6.2 Códigos de resposta e tratamento de exceções

Toda falha devolve o mesmo formato (`ErroResposta`). A camada `exception` usa polimorfismo: cada subclasse de `ApiException` informa o próprio status em `getStatus()`, e o `GlobalExceptionHandler` (`@RestControllerAdvice`) trata também os erros do Spring (validação, JSON inválido, rota inexistente, banco indisponível).

| Status | Origem | Quando ocorre |
|---|---|---|
| 400 | Bean Validation, JSON inválido, parâmetro com tipo errado | Campos obrigatórios ausentes, formato de e-mail/CEP/CNPJ/telefone, data fora do padrão ISO, valor de enum inexistente |
| 401 | `CredenciaisInvalidasException` | Login com e-mail ou senha incorretos |
| 404 | `RecursoNaoEncontradoException`, rota inexistente | Id não cadastrado ou endpoint errado |
| 405 | Método não suportado | Ex.: PATCH em um endpoint que não o aceita |
| 409 | `ConflitoException` | E-mail ou CNPJ já cadastrado, solicitação ativa duplicada, retirada já agendada, remoção de registro com histórico |
| 422 | `RegraNegocioException` | Segurança alimentar, quantidade acima do disponível, status que não permite a ação, horário fora da janela de retirada |
| 500 / 503 | `BancoDeDadosException`, erros inesperados | Falha de acesso ao Oracle (detalhes técnicos ficam só no log) |

```
422 Unprocessable Entity
{ "timestamp": "2026-11-13T14:48:17", "status": 422, "erro": "Unprocessable Entity",
  "mensagem": "Alimento não publicado: corrija as pendências de segurança alimentar.",
  "caminho": "/api/alimentos",
  "detalhes": [ "É obrigatório declarar ciência das orientações de segurança alimentar
(declaracaoSeguranca = true).",
                "Itens da categoria 'Laticínios' precisam estar refrigerados ou congelados para
serem doados." ] }
```

# 7. Modelo de dados (DER)

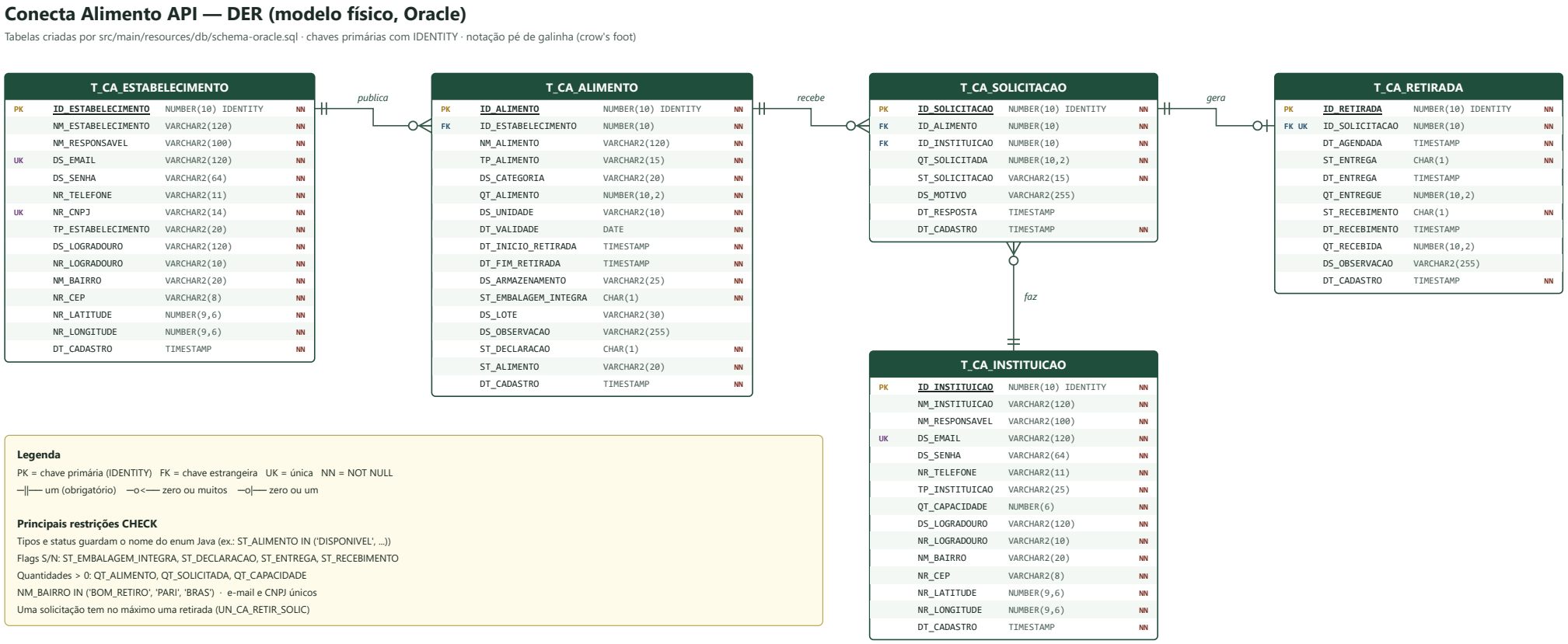



Figura 4 - DER das cinco tabelas no Oracle. Cada tabela corresponde a uma classe do pacote model; os enums são gravados pelo nome (VARCHAR2) e os booleanos como 'S'/'N' (CHAR(1)).

| Tabela | Classe Java | DAO | Relacionamentos |
|---|---|---|---|
| `T_CA_ESTABELECIMENTO` | EstabelecimentoDoador | EstabelecimentoDAO | 1:N com T_CA_ALIMENTO |
| `T_CA_INSTITUICAO` | InstituicaoReceptora | InstituicaoDAO | 1:N com T_CA_SOLICITACAO |
| `T_CA_ALIMENTO` | AlimentoPerecivel / AlimentoNaoPerecivel (coluna TP_ALIMENTO) | AlimentoDAO | N:1 estabelecimento; 1:N solicitações |
| `T_CA_SOLICITACAO` | Solicitacao | SolicitacaoDAO | N:1 alimento e instituição; 1:0..1 retirada |
| `T_CA_RETIRADA` | Retirada | RetiradaDAO | 1:1 solicitação (FK única) |

# 8. Instruções de execução

## 8.1 Ferramentas e versões

| Ferramenta | Versão / uso |
|---|---|
| Java (JDK) | 21 |
| Spring Boot | 3.5.16 (Web, JDBC, Validation) |
| Banco de dados | Oracle Database 19c da FIAP ( `oracle.fiap.com.br:1521:ORCL` ), driver `ojdbc11` |
| Documentação da API | springdoc-openapi 2.8.17 (Swagger UI) |
| Build | Maven 3.9 (embutido no IntelliJ) |
| IDE | IntelliJ IDEA |
| Deploy | Render.com (Web Service com Docker: Maven + JDK 21 no build, JRE 21 na execução) |
| Testes manuais | Swagger UI e arquivo `requisicoes.http` (HTTP Client do IntelliJ) |

## 8.2 Links do projeto

| | |
|---|---|
| **Repositório GitHub** | https://github.com/Guilherme12376/GS-Java |
| **Deploy (Render)** | Não publicado — a API roda localmente (seção 8); o projeto está pronto para deploy no Render via Dockerfile e render.yaml |
| **Swagger (documentação interativa)** | http://localhost:8080/swagger-ui.html (com a API em execução local) |

> No plano gratuito do Render o serviço hiberna após 15 minutos sem acesso; a primeira requisição pode levar cerca de 1 minuto para acordá-lo.

## 8.3 Executar localmente no IntelliJ

1. Instale o **JDK 21** e o **IntelliJ IDEA**.
2. Clone o repositório: `git clone https://github.com/Guilherme12376/GS-Java` (ou extraia o código do .zip).
3. No IntelliJ, **File › Open** e selecione a pasta do projeto; o `pom.xml` é reconhecido e o Maven baixa as dependências.
4. Em **File › Project Structure › Project**, selecione o **SDK 21**.
5. Abra `ConectaAlimentoApplication` e clique no ▶ ao lado do método `main`.
6. Em **Run › Edit Configurations › Environment variables**, informe as credenciais do Oracle da FIAP: `DB_USER=rmXXXXX;DB_PASSWORD=sua_senha` (opcional: `DB_URL`). As credenciais nunca ficam no código.
7. Execute. Na primeira subida a API cria as tabelas `T_CA_*` e, com o banco vazio, carrega dados de demonstração. O script manual equivalente está em `database/script-oracle.sql`.
8. Acesse `http://localhost:8080/swagger-ui.html`. Em `/api/status` o campo `"banco": "UP"` confirma a conexão com o Oracle.

> Sem acesso ao Oracle (ex.: fora da rede da FIAP), é possível rodar com o perfil `h2` (banco em memória): em **Edit Configurations › Active profiles** informe `h2`, ou `mvn spring-boot:run -Dspring-boot.run.profiles=h2`.

## 8.4 Deploy no Render

1. Publique o projeto no GitHub (a raiz do repositório contém `pom.xml`, `Dockerfile` e `render.yaml`).
2. No render.com: **New › Web Service** e conecte o repositório.
3. **Language: Docker** (o Render usa o `Dockerfile` do projeto) e **Instance type: Free**.
4. Em **Environment Variables**: `DB_URL=jdbc:oracle:thin:@oracle.fiap.com.br:1521:ORCL`, `DB_USER` e `DB_PASSWORD` com as credenciais do Oracle.
5. Em **Advanced › Health Check Path**: `/`. Clique em **Create Web Service** e aguarde o build.
6. Teste a URL pública em `/api/status` e `/swagger-ui.html`. (Alternativa: **New › Blueprint** com o arquivo `render.yaml`.)

## 8.5 Como testar

Pelo Swagger (ou `requisicoes.http`), siga o fluxo principal com os dados de demonstração (todos fictícios, senha `123456`):

| Usuário de demonstração | Perfil |
|---|---|
| `padaria@demo.com`, `mercado@demo.com`, `hortifruti@demo.com` | Estabelecimentos doadores (Pari, Brás, Bom Retiro) |
| `cozinha@demo.com`, `ong@demo.com` | Instituições receptoras (Bom Retiro, Brás) |

1. `POST /api/login` com um dos usuários acima.
2. `POST /api/alimentos` para publicar uma doação (ou use uma das já disponíveis).
3. `GET /api/alimentos/disponiveis?instituicaoId=1` para ver as ofertas com a distância.
4. `POST /api/solicitacoes` → `PUT /api/solicitacoes/{id}/aceitar`.
5. `POST /api/retiradas` → `PUT /api/retiradas/{id}/confirmar-entrega` → `PUT /api/retiradas/{id}/confirmar-recebimento`.
6. `GET /api/indicadores` mostra a doação concluída nos números de impacto.

Durante o desenvolvimento, os endpoints foram exercitados localmente (perfil h2): CRUDs, o fluxo completo da doação e os cenários de erro 400, 401, 404, 405, 409 e 422.